

AI EXAM

TALENT INTERNSHIP PROGRAM

TEST NO.: 101

Problem 01: Calorie Counting (40 Points)

--- Part One ---

Santa's reindeer typically eat regular reindeer food, but they need a lot of magical energy to deliver presents on Christmas. For that, their favorite snack is a special type of star fruit that only grows deep in the jungle. The Elves have brought you on their annual expedition to the grove where the fruit grows.

To supply enough magical energy, the expedition needs to retrieve a minimum of fifty stars by December 25th. Although the Elves assure you that the grove has plenty of fruit, you decide to grab any fruit you see along the way, just in case.

Collect stars by solving puzzles. Two puzzles will be made available on each day in the Advent calendar; the second puzzle is unlocked when you complete the first. Each puzzle grants one star. Good luck!

The jungle must be too overgrown and difficult to navigate in vehicles or access from the air; the Elves' expedition traditionally goes on foot. As your boats approach land, the Elves begin taking inventory of their supplies. One important consideration is food - in particular, the number of Calories each Elf is carrying (your puzzle input).

The Elves take turns writing down the number of Calories contained by the various meals, snacks, rations, etc. that they've brought with them, one item per line. Each Elf separates their own inventory from the previous Elf's inventory (if any) by a blank line.

For example, suppose the Elves finish writing their items' Calories and end up with the following list:

```
1000
2000
3000
```

4000
5000
6000
7000
8000
9000
10000

This list represents the Calories of the food carried by five Elves:

The first Elf is carrying food with 1000, 2000, and 3000 Calories, a total of 6000 Calories.

The second Elf is carrying one food item with 4000 Calories.

The third Elf is carrying food with 5000 and 6000 Calories, a total of 11000 Calories.

The fourth Elf is carrying food with 7000, 8000, and 9000 Calories, a total of 24000 Calories.

The fifth Elf is carrying one food item with 10000 Calories.

In case the Elves get hungry and need extra snacks, they need to know which Elf to ask: they'd like to know how many Calories are being carried by the Elf carrying the most Calories. In the example above, this is 24000 (carried by the fourth Elf).

Find the Elf carrying the most Calories. **How many total Calories is that Elf carrying?**

--- Part Two ---

By the time you calculate the answer to the Elves' question, they've already realized that the Elf carrying the most Calories of food might eventually run out of snacks.

To avoid this unacceptable situation, the Elves would instead like to know the total Calories carried by the top three Elves carrying the most Calories. That way, even if one of those Elves runs out of snacks, they still have two backups.

In the example above, the top three Elves are the fourth Elf (with 24000 Calories), then the third Elf (with 11000 Calories), then the fifth Elf (with 10000 Calories). The sum of the Calories carried by these three elves is 45000.

Find the top three Elves carrying the most Calories. **How many Calories are those Elves carrying in total?**

Problem 02: Tuning Trouble (40 Points)

--- Part One ---

The preparations are finally complete; you and the Elves leave camp on foot and begin to make your way toward the star fruit grove.

As you move through the dense undergrowth, one of the Elves gives you a handheld device. He says that it has many fancy features, but the most important one to set up right now is the communication system.

However, because he's heard you have significant experience dealing with signal-based systems, he convinced the other Elves that it would be okay to give you their one malfunctioning device - surely you'll have no problem fixing it.

As if inspired by comedic timing, the device emits a few colorful sparks.

To be able to communicate with the Elves, the device needs to lock on to their signal. The signal is a series of seemingly-random characters that the device receives one at a time.

To fix the communication system, you need to add a subroutine to the device that detects a start-of-packet marker in the datastream. In the protocol being used by the Elves, the start of a packet is indicated by a sequence of four characters that are all different.

The device will send your subroutine a datastream buffer (your puzzle input); your subroutine needs to identify the first position where the four most recently received characters were all different. Specifically, it needs to report the number of characters from the beginning of the buffer to the end of the first such four-character marker.

For example, suppose you receive the following datastream buffer:

```
mjqjpqmgbljsphdztnvjfqwrcgsmlb
```

After the first three characters (mjq) have been received, there haven't been enough characters received yet to find the marker. The first time a marker could occur is after the

fourth character is received, making the most recent four characters mj~~qj~~. Because j is repeated, this isn't a marker.

The first time a marker appears is after the seventh character arrives. Once it does, the last four characters received are j~~pqm~~, which are all different. In this case, your subroutine should report the value 7, because the first start-of-packet marker is complete after 7 characters have been processed.

Here are a few more examples:

- bvwbjplbgvbhserlpqdmjqwftvncz: first marker after character 5
- nppdvjthqldpwnccszvftbrmjlhg: first marker after character 6
- nzrnrfnfntjfmvfwzdfjlvtnbhcprrsg: first marker after character 10
- zcfzfwzzqfrrljwzlrfrnpqdbhtmscgvjw: first marker after character 11

How many characters need to be processed before the first start-of-packet marker is detected?

--- Part Two ---

Your device's communication system is correctly detecting packets, but still isn't working. It looks like it also needs to look for messages.

A start-of-message marker is just like a start-of-packet marker, except it consists of 14 distinct characters rather than 4.

Here are the first positions of start-of-message markers for all of the above examples:

- mjqqjpqmgbljsphdztnvjfqwrcgsmlb: first marker after character 19
- bvwbjplbgvbhserlpqdmjqwftvncz: first marker after character 23
- nppdvjthqldpwnccszvftbrmjlhg: first marker after character 23
- nzrnrfnfntjfmvfwzdfjlvtnbhcprrsg: first marker after character 29
- zcfzfwzzqfrrljwzlrfrnpqdbhtmscgvjw: first marker after character 26

How many characters need to be processed before the first start-of-message marker is detected?

Problem 03: Beacon Exclusion Zone (20 Points)

--- Part One ---

You feel the ground rumble again as the distress signal leads you to a large network of subterranean tunnels. You don't have time to search them all, but you don't need to: your pack contains a set of deployable sensors that you imagine were originally built to locate lost Elves.

The sensors aren't very powerful, but that's okay; your handheld device indicates that you're close enough to the source of the distress signal to use them. You pull the emergency sensor system out of your pack, hit the big button on top, and the sensors zoom off down the tunnels.

Once a sensor finds a spot it thinks will give it a good reading, it attaches itself to a hard surface and begins monitoring for the nearest signal source beacon. Sensors and beacons always exist at integer coordinates. Each sensor knows its own position and can determine the position of a beacon precisely; however, sensors can only lock on to the one beacon closest to the sensor as measured by the Manhattan distance. (There is never a tie where two beacons are the same distance to a sensor.)

It doesn't take long for the sensors to report back their positions and closest beacons (your puzzle input). For example:

```
Sensor at x=2, y=18: closest beacon is at x=-2, y=15
Sensor at x=9, y=16: closest beacon is at x=10, y=16
Sensor at x=13, y=2: closest beacon is at x=15, y=3
Sensor at x=12, y=14: closest beacon is at x=10, y=16
Sensor at x=10, y=20: closest beacon is at x=10, y=16
Sensor at x=14, y=17: closest beacon is at x=10, y=16
Sensor at x=8, y=7: closest beacon is at x=2, y=10
Sensor at x=2, y=0: closest beacon is at x=2, y=10
Sensor at x=0, y=11: closest beacon is at x=2, y=10
Sensor at x=20, y=14: closest beacon is at x=25, y=17
Sensor at x=17, y=20: closest beacon is at x=21, y=22
Sensor at x=16, y=7: closest beacon is at x=15, y=3
Sensor at x=14, y=3: closest beacon is at x=15, y=3
Sensor at x=20, y=1: closest beacon is at x=15, y=3
```

So, consider the sensor at 2,18; the closest beacon to it is at -2,15. For the sensor at 9,16, the closest beacon to it is at 10,16.

Drawing sensors as S and beacons as B, the above arrangement of sensors and beacons looks like this:

	0	5	1	1	2	2
	0	5	0	5	0	5
0	...	S	...	...	...	...
1	...	...	...	...	S	...
2	...	...	...	S	...	...
3	...	...	...	SB	...	...
4	...	...	...	...	...	...
5	...	...	...	...	...	...
6	...	...	...	...	...	...
7	...	...	S	...	S	...
8	...	...	...	...	...	...
9	...	...	...	...	...	...
10	...	B	...	...	...	...
11	...	S	...	...	...	...
12	...	...	...	...	...	...
13	...	...	...	...	...	...
14	...	...	...	S	...	S
15	B	...	...	...	...	...
16	...	...	...	SB	...	...
17	...	...	...	S	...	B
18	...	S	...	...	...	...
19	...	...	...	...	...	...
20	...	...	S	...	S	...
21	...	...	...	...	...	...
22	...	...	...	...	...	B

This isn't necessarily a comprehensive map of all beacons in the area, though. Because each sensor only identifies its closest beacon, if a sensor detects a beacon, you know there are no other beacons that close or closer to that sensor. There could still be beacons that just happen to not be the closest beacon to any sensor. Consider the sensor at 8,7:

	0	5	1	1	2	2
	0	5	0	5	0	5
-2	...	...	#	...	...	...
-1	...	...	###	...	...	...
0	...	S	#####	...	...	...
1	...	...	#####	...	S	...
2	...	...	#####	S	...	...

3	#####SB.....
4	#####.....
5	...#####.....
6	..#####.....
7	.#####S#####S#.....
8	..#####.....
9	...#####.....
10	B#####.....
11	..S..#####.....
12	#####.....
13	#####.....
14	#####.S.....S.....
15	B.....###.....
16	#SB.....
17	S.....B.....
18	S.....
19	
20	S.....S.....
21	
22	B.....

This sensor's closest beacon is at 2,10, and so you know there are no beacons that close or closer (in any positions marked #).

None of the detected beacons seem to be producing the distress signal, so you'll need to work out where the distress beacon is by working out where it isn't. For now, keep things simple by counting the positions where a beacon cannot possibly be along just a single row.

So, suppose you have an arrangement of beacons and sensors like in the example above and, just in the row where $y=10$, you'd like to count the number of positions a beacon cannot possibly exist. The coverage from all sensors near that row looks like this:

			1	1	2	2
	0	5	0	5	0	5
9	#####.....					
10	..#####B#####.....					
11	.#####S#####.#####.					

In this example, in the row where $y=10$, there are 26 positions where a beacon cannot be present.

Consult the report from the sensors you just deployed. **In the row where $y=2000000$, how many positions cannot contain a beacon?**

--- Part Two ---

Your handheld device indicates that the distress signal is coming from a beacon nearby. The distress beacon is not detected by any sensor, but the distress beacon must have x and y coordinates each no lower than 0 and no larger than 4000000.

To isolate the distress beacon's signal, you need to determine its *tuning frequency*, which can be found by multiplying its x coordinate by 4000000 and then adding its y coordinate.

In the example above, the search space is smaller: instead, the x and y coordinates can each be at most 20. With this reduced search area, there is only a single position that could have a beacon: $x=14$, $y=11$. The tuning frequency for this distress beacon is 56000011.

Find the only possible position for the distress beacon. **What is its tuning frequency?**

End of test paper.